

Sistemas Operativos

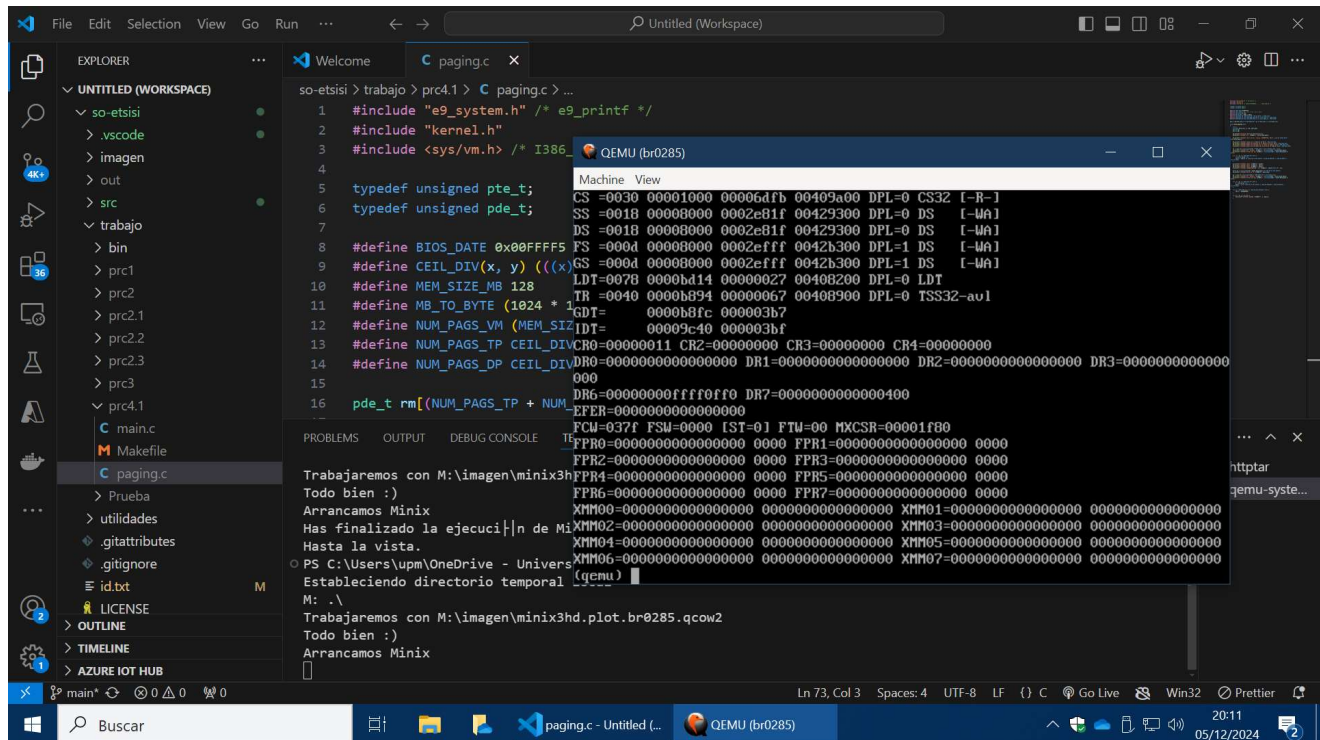
Práctica 4.2

Fecha: 5/12/2024

Alumno: Yzan Martinez Manzano

Correo: yzan.martinez.manzano@alumnos.upm.es

1. Registros CR0 y CR3



```
1 #include "e9_system.h" /* e9_printf */
2 #include "kernel.h"
3 #include <sys/vm.h> /* I386_
4
5 typedef unsigned pte_t;
6 typedef unsigned pde_t;
7
8 #define BIOS_DATE 0x00FFFF5
9 #define CEIL_DIV(x, y) (((x)+
10 #define MEM_SIZE_MB 128
11 #define MB_TO_BYTE (1024 * 1
12 #define NUM_PAGS_VM (MEM_SIZE
13 #define NUM_PAGS_TP CEIL_DIV(CR0=00000011 CR2=00000000 CR3=00000000 CR4=00000000
14 #define NUM_PAGS_DP CEIL_DIV(DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
15
16 pde_t rm[(NUM_PAGS_TP + NUM
17
18 PROBLEMS OUTPUT DEBUG CONSOLE
19
20 Trabajaremos con M:\imagen\minix3h
21 Todo bien :)
22 Arrancamos Minix
23 Has finalizado la ejecuci|n de Mi
24 Hasta la vista.
25 PS C:\Users\upm\OneDrive - Univers
26 Estableciendo directorio temporal (qemu)
27 M: .\
28 Trabajaremos con M:\imagen\minix3hd.plot.br0285.qcow2
29 Todo bien :)
30 Arrancamos Minix
```

Machine View

Register	Value
CS	00001000 00006dfb 00409a00 DPL=0 CS32 [-R-]
SS	0001B 0000B000 0002e81f 00429300 DPL=0 DS [-WA]
DS	0001B 0000B000 0002e81f 00429300 DPL=0 DS [-WA]
FS	0000d 0000B000 0002efff 0042b300 DPL=1 DS [-WA]
GS	0000d 0000B000 0002efff 0042b300 DPL=1 DS [-WA]
LDT	0007B 0000bd14 00000027 00408200 DPL=0 LDT
TR	00040 0000b894 00000067 00408900 DPL=0 TSS32-aui
GDT	0000b8fc 000003b7
IDT	00009c40 000003bf

Trabajaremos con M:\imagen\minix3hd.plot.br0285.qcow2
Todo bien :)
Arrancamos Minix

2. Contenido DP

The screenshot shows a Visual Studio Code editor with a workspace named 'Untitled (Workspace)'. The Explorer sidebar on the left shows a project structure with folders like 'so-etsisi', 'imagen', 'src', and 'trabajo'. The 'trabajo' folder is expanded, showing subfolders 'bin', 'prc1', 'prc2', 'prc2.1', 'prc2.2', 'prc2.3', and 'prc4.1'. The 'prc4.1' folder is selected, showing files 'main.c', 'Makefile', and 'paging.c'. The 'paging.c' file is open in the editor, showing C code for memory management, including macros for BIOS date, memory size, and page definitions. A QEMU terminal window is open in the foreground, displaying the QEMU machine view, which shows various registers and memory addresses. The terminal output includes the QEMU command line and the resulting machine state.

```
so-etsisi > trabajo > prc4.1 > C paging.c ...
1 #include "e9_system.h" /* e9_printf */
2 #include "kernel.h"
3 #include <sys/vm.h> /* I386_VM_PRESENT
4
5 typedef unsigned pte_t;
6 typedef unsigned pde_t;
7
8 #define BIOS_DATE 0x00FFFF5
9 #define CEIL_DIV(x, y) (((x) + (y) - 1) / y)
10 #define MEM_SIZE_MB 128
11 #define MB_TO_BYTE (1024 * 1024)
12 #define NUM_PAGS_VM (MEM_SIZE_MB * MB_TO_BYTE)
13 #define NUM_PAGS_TP CEIL_DIV(NUM_PAGS_VM, PAGE_SIZE)
14 #define NUM_PAGS_DP CEIL_DIV(NUM_PAGS_TP, PAGE_SIZE)
15
16 pde_t rm[(NUM_PAGS_TP + NUM_PAGS_DP) * PAGE_SIZE]
```

QEMU (br0285)

Machine View

```
TR =0040 0000b894 00000067 00408900 DPL=0 TSS32-au1
GDT= 0000b8fc 000003b7
IDT= 00009c40 000003bf
CR0=00000011 CR2=00000000 CR3=00000000 CR4=00000000
DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
DR6=00000000ffff0fff DR7=00000000000000400
EFER=0000000000000000
FCW=037f FSW=0000 IST=01 FTW=00 MXCSR=00001f80
FPR0=0000000000000000 0000 FPR1=0000000000000000 0000
FPR2=0000000000000000 0000 FPR3=0000000000000000 0000
FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=0000000000000000 0000
XMM00=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
(qemu) x /20xww 0x00000000
0000000000000000: 0xf00ff53 0xf00ff53 0xf00e2c3 0xf00ff53
0000000000000010: 0xf00ff53 0xf00ff54 0xf00ff53 0xf00ff53
0000000000000020: 0xf00fea5 0xf00e987 0xf00d43d 0xf00d43d
0000000000000030: 0xf00d43d 0xf00d43d 0xf00ef57 0xf00d43d
0000000000000040: 0xc005606 0xf00f84d 0xf00f841 0xf00e3fe
(qemu)
```

The screenshot shows the QEMU (br0285) Machine View window. It displays the state of the QEMU machine, including registers and memory addresses. The output is as follows:

```
QEMU (br0285)
Machine View
TR =0040 0000b894 00000067 00408900 DPL=0 TSS32-au1
GDT= 0000b8fc 000003b7
IDT= 00009c40 000003bf
CR0=00000011 CR2=00000000 CR3=00000000 CR4=00000000
DR0=0000000000000000 DR1=0000000000000000 DR2=0000000000000000 DR3=0000000000000000
DR6=00000000ffff0fff DR7=00000000000000400
EFER=0000000000000000
FCW=037f FSW=0000 IST=01 FTW=00 MXCSR=00001f80
FPR0=0000000000000000 0000 FPR1=0000000000000000 0000
FPR2=0000000000000000 0000 FPR3=0000000000000000 0000
FPR4=0000000000000000 0000 FPR5=0000000000000000 0000
FPR6=0000000000000000 0000 FPR7=0000000000000000 0000
XMM00=0000000000000000 0000000000000000 XMM01=0000000000000000 0000000000000000
XMM02=0000000000000000 0000000000000000 XMM03=0000000000000000 0000000000000000
XMM04=0000000000000000 0000000000000000 XMM05=0000000000000000 0000000000000000
XMM06=0000000000000000 0000000000000000 XMM07=0000000000000000 0000000000000000
(qemu) x /20xww 0x00000000
0000000000000000: 0xf00ff53 0xf00ff53 0xf00e2c3 0xf00ff53
0000000000000010: 0xf00ff53 0xf00ff54 0xf00ff53 0xf00ff53
0000000000000020: 0xf00fea5 0xf00e987 0xf00d43d 0xf00d43d
0000000000000030: 0xf00d43d 0xf00d43d 0xf00ef57 0xf00d43d
0000000000000040: 0xc005606 0xf00f84d 0xf00f841 0xf00e3fe
(qemu)
```

0xf000ff53, 0xf000ff53, 0xf000e2c3, 0xf000ff53, 0xf000ff54, 0xf000ff53, 0xf000ff53, 0xf000fea5, 0xf000e97, 0xf0004d43, 0xf0004d43, 0xf0004d43, 0xf000ef57, 0xf0004d43, 0xf0005f64, 0xf000f841, 0xf000e3fe

Cada entrada tiene 32 bits, que representan diferentes configuraciones relacionadas con la gestión de memoria. Aquí abajo dejo una tabla de internet de cómo es y como se describe.

Bits	Descripción
------	-------------

- | | |
|-------|--|
| 0 | Presente (1 si la entrada es válida). |
| 1 | Read/Write (1 permite escritura). |
| 2 | User/Supervisor (1 accesible a usuarios). |
| 3 | Page-level write-through (1 habilita write-through). |
| 4 | Cache Disable (1 desactiva caché). |
| 5 | Accessed (1 si se accedió a esta entrada). |
| 6 | Dirty (usado en PTE para páginas modificadas). |
| 7 | Tamaño de página (1 si es de 4 MiB). |
| 8–11 | Reservado (deben ser 0). |
| 12–31 | Dirección base (marco de página alineado a 4 KiB). |

Cada entrada tiene un valor en hexadecimal, que se puede descomponer para ver qué hace. Este valor tiene información sobre:

- Si la entrada está activa (bit 0).
- Si se permite escribir (bit 1).
- Si es accesible solo para el sistema operativo (bit 2).
- Si usa caché o no (bits 3 y 4).
- Y lo más importante: a qué parte de la memoria física apunta (bits 12 al 31).

-Entradas repetidas: 0xf000ff53

Binario: 1111 0000 0000 0000 1111 1111 0101 0011

Bit 0: 1 → La entrada está presente.

Bit 1: 1 → Permite escritura.

Bit 2: 0 → Solo el sistema operativo puede acceder.

Bit 3: 1 → Write-through habilitado.

Bit 4: 0 → Caché habilitado.

Bit 5: 1 → Entrada marcada como accedida.

Bits 12–31: 0xf000 → Dirección base del marco.

Estas configuraciones son comunes y se repiten en varias entradas, lo que sugiere que se usan para páginas del sistema con permisos similares.

-Entrada 0xf000e2c3

Binario: 1111 0000 0000 0000 1110 0010 1100 0011

Bit 0: 1 → Entrada activa.

Bit 1: 1 → Escritura permitida.

Bit 2: 0 → Supervisor.

Bits 12–31: 0xf000 → Marco diferente al resto.

-Entrada 0xf000fea5

- **Binario:** 1111 0000 0000 0000 1111 1110 1010 0101
- **Bit 0:** 1 → Activa.
- **Bit 1:** 1 → Permite escritura.
- **Bit 2:** 0 → Solo supervisor.
- **Bits 12–31:** Dirección física diferente (0xf000).

-Entradas 0xf0004d43

- **Binario:** 1111 0000 0000 0000 0100 1101 0100 0011
- Similar a las anteriores, pero apunta a otro marco (0xf000).

-Entradas específicas con diferencias:

0xf000ef57:

- Similar a las anteriores, pero los bits 12–31 cambian (dirección distinta).

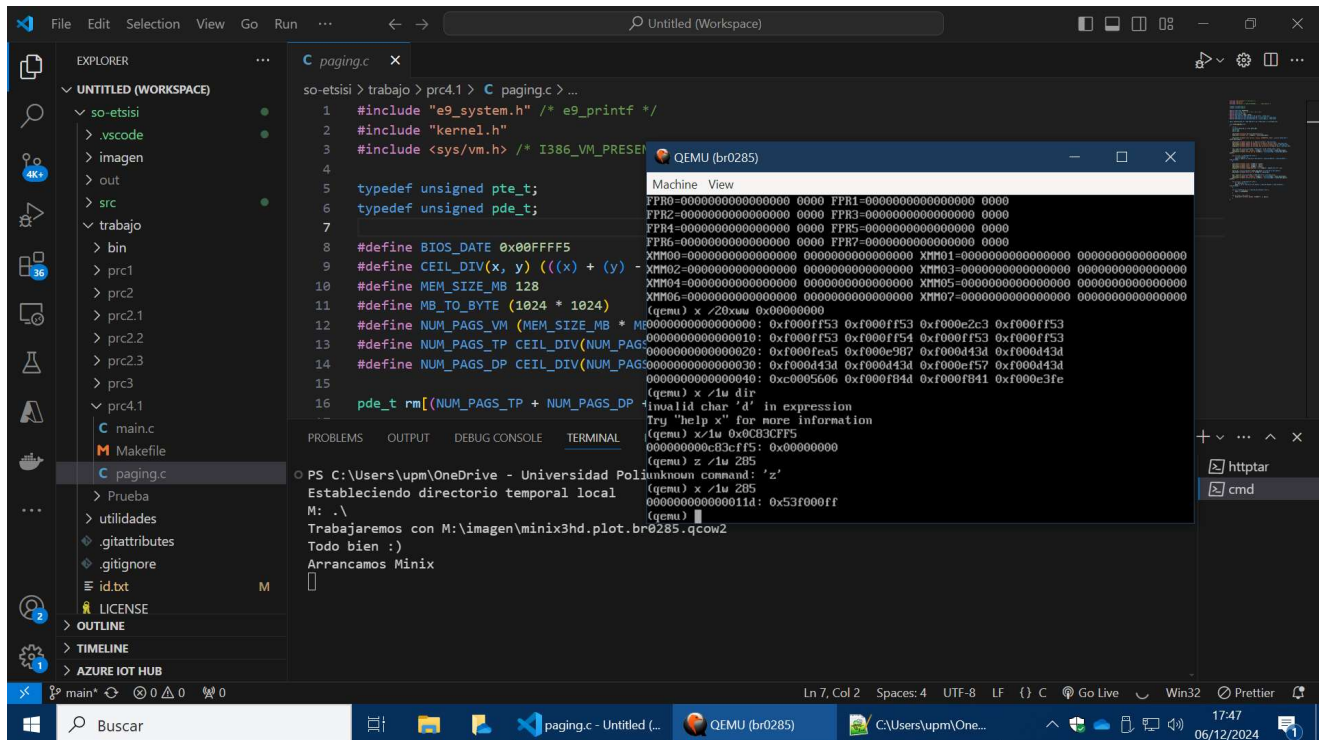
0xf0005f64:

- Apunta a otro marco (bits 12–31 diferentes).

0xf000e3fe:

- Dirección base y configuración distinta, posiblemente para otro uso.

3. Análisis de una PTE

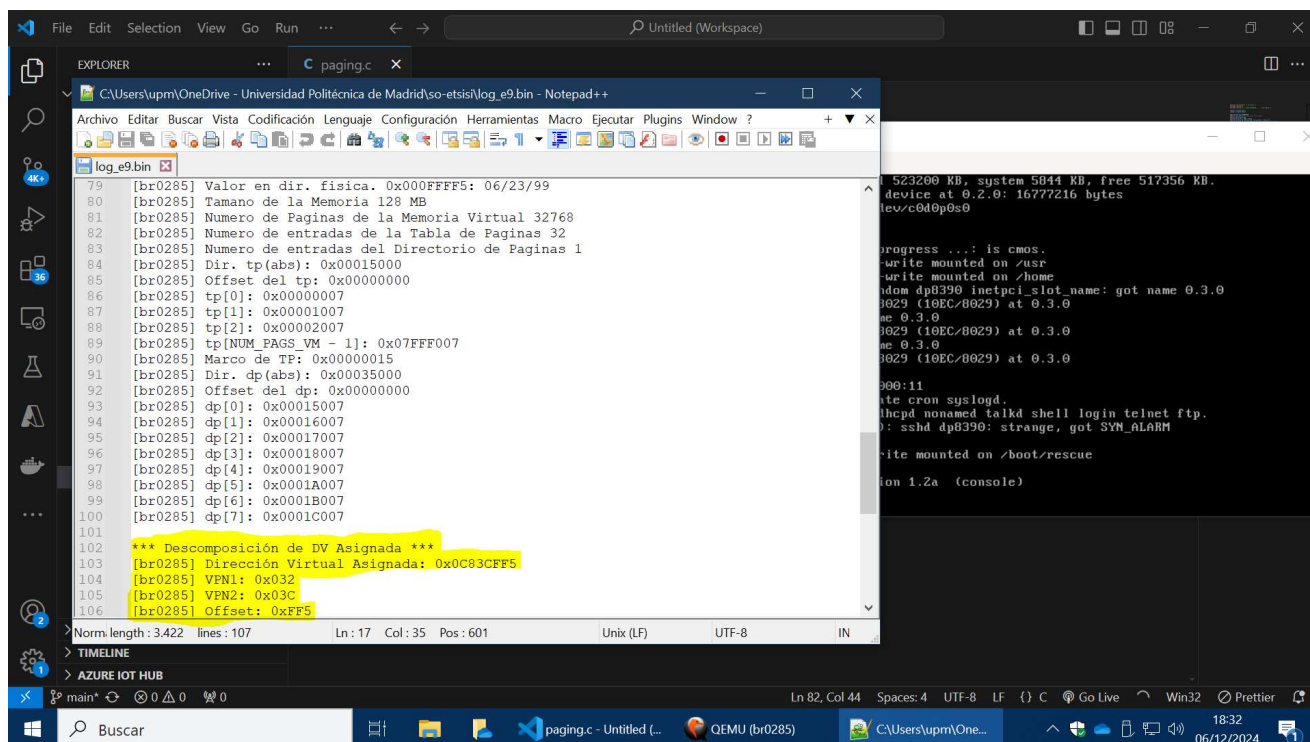


Dirección de memoria 0x53f000ff

-Binario: 0101 0011 1111 0000 0000 0000 1111 1111

- 1 → La entrada está activa.
- 1 → Escritura habilitada.
- 1 → Acceso permitido para usuarios.
- 1 → Write-through habilitado.
- 0 → Caché habilitada.
- 1 → La página fue accedida.
- 1 → La página fue modificada.
- Reservados (0 en este caso).
- 0x53f00 → Dirección física base del marco de memoria alineado a 4 KiB.

4. Campos de la DV asignada



```
#include "e9_system.h" /* e9_printf */
#include "kernel.h"
#include <sys/vm.h> /* I386_VM_PRESENT, ... , I386_CR0_PG */

typedef unsigned pte_t;
typedef unsigned pde_t;

#define BIOS_DATE 0x00FFFF5
#define CEIL_DIV(x, y) (((x) + (y) - 1) / (y))
#define MEM_SIZE_MB 128
#define MB_TO_BYTE (1024 * 1024)
#define NUM_PAGS_VM (MEM_SIZE_MB * MB_TO_BYTE) / PAGE_SIZE
#define NUM_PAGS_TP CEIL_DIV(NUM_PAGS_VM * sizeof(pte_t), PAGE_SIZE)
#define NUM_PAGS_DP CEIL_DIV(NUM_PAGS_TP * sizeof(pde_t), PAGE_SIZE)
#define I386_VM_PT_ENT_SHIFT 12
#define I386_VM_PD_ENT_SHIFT 22
#define I386_VM_PT_ENT_MASK 0x3FF
#define I386_VM_OFFSET_MASK 0xFFF

pde_t rm[(NUM_PAGS_TP + NUM_PAGS_DP + 1) * PAGE_SIZE / sizeof(pde_t)];

void setup_paging(void)
{
    int i;
    unsigned marco_tp, m, cr0, marco_dp;
    unsigned dv_asignada = 0x0C83CFF5;
    unsigned vpn1, vpn2, offset;
    pde_t* tp;
    pde_t* dp;
```

```

e9_printf("\n***Yzan Martinez Manzano***\n");
e9_printf("[br0285] Dir: 0x%08X\n", kinfo.data_base);

e9_printf("[br0285] Valor en dir. fisica. 0x000FFFF5: %s\n", (char*) (BIOS_DATE -
kinfo.data_base));

e9_printf("[br0285] Tamano de la Memoria %u MB\n", MEM_SIZE_MB);
e9_printf("[br0285] Numero de Paginas de la Memoria Virtual %u\n", NUM_PAGS_VM);
e9_printf("[br0285] Numero de entradas de la Tabla de Paginas %u\n", NUM_PAGS_TP);
e9_printf("[br0285] Numero de entradas del Directorio de Paginas %u\n", NUM_PAGS_DP);

tp = (pde_t*) (((unsigned)rm) + PAGE_SIZE - 1) & (~(PAGE_SIZE - 1));
e9_printf("[br0285] Dir. tp(abs): 0x%08X\n", ((unsigned)tp) + kinfo.data_base);
e9_printf("[br0285] Offset del tp: 0x%08X\n", ((unsigned)tp) + kinfo.data_base &
0xFFFF);

for (i = 0; i < NUM_PAGS_VM; i++) {
    unsigned marco = i;
    tp[i] = (marco << I386_VM_PT_ENT_SHIFT) | I386_VM_PRESENT | I386_VM_WRITE |
I386_VM_USER;
}

e9_printf("[br0285] tp[0]: 0x%08X\n", tp[0]);
e9_printf("[br0285] tp[1]: 0x%08X\n", tp[1]);
e9_printf("[br0285] tp[2]: 0x%08X\n", tp[2]);
e9_printf("[br0285] tp[NUM_PAGS_VM - 1]: 0x%08X\n", tp[NUM_PAGS_VM - 1]);

marco_tp = (((unsigned)tp + kinfo.data_base) >> I386_VM_PT_ENT_SHIFT);
e9_printf("[br0285] Marco de TP: 0x%08X\n", marco_tp);

dp = (pde_t*) (((unsigned)tp) + (NUM_PAGS_VM * sizeof(pte_t)));
e9_printf("[br0285] Dir. dp(abs): 0x%08X\n", ((unsigned)dp) + kinfo.data_base);
e9_printf("[br0285] Offset del dp: 0x%08X\n", ((unsigned)dp) + kinfo.data_base &
0xFFFF);

for (i = 0; i < NUM_PAGS_TP; i++) {
    m = marco_tp + i;
    dp[i] = (m << I386_VM_PT_ENT_SHIFT) | I386_VM_PRESENT | I386_VM_WRITE |
I386_VM_USER;
}

for (i = NUM_PAGS_TP; i < I386_VM_DIR_ENTRIES; i++) {
    dp[i] = 0x00000000;
}

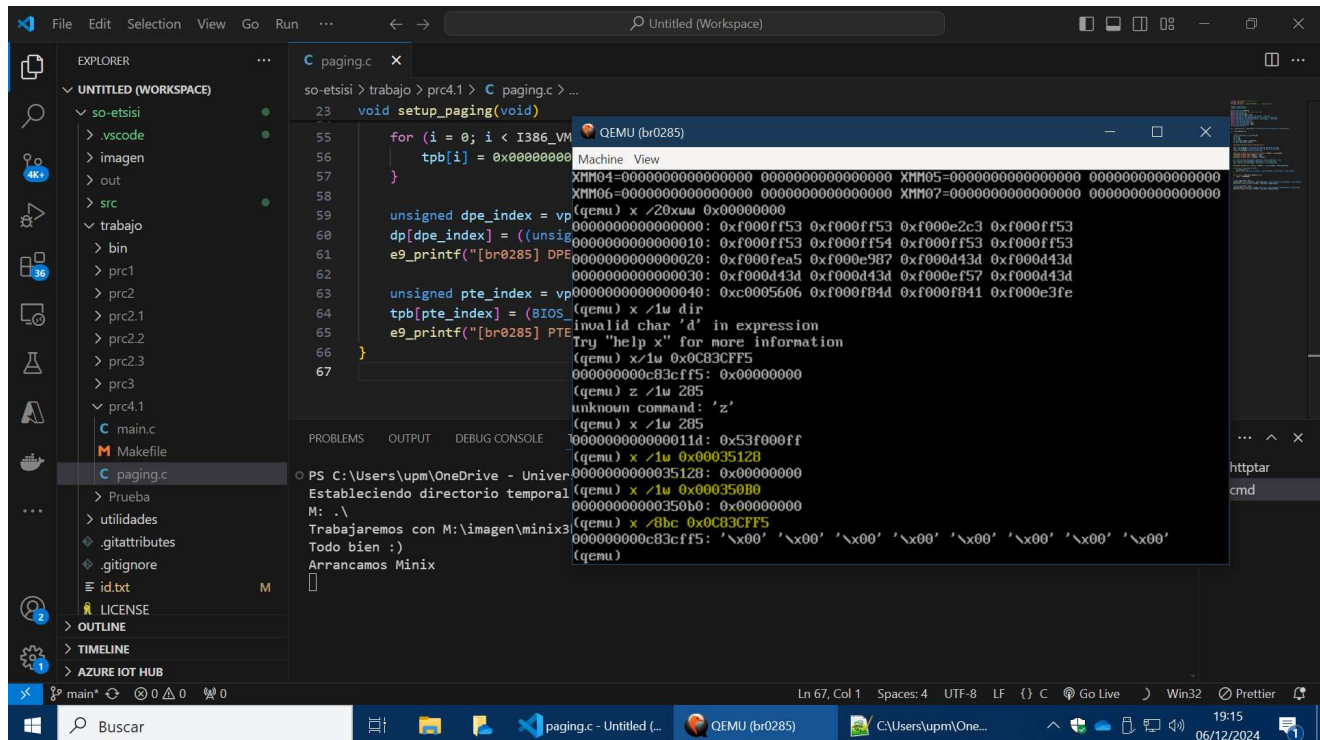
for (i = 0; i < 8; i++) {
    e9_printf("[br0285] dp[%d]: 0x%08X\n", i, dp[i]);
}

vpn1 = (dv_asignada >> I386_VM_PD_ENT_SHIFT) & I386_VM_PT_ENT_MASK;
vpn2 = (dv_asignada >> I386_VM_PT_ENT_SHIFT) & I386_VM_PT_ENT_MASK;
offset = dv_asignada & I386_VM_OFFSET_MASK;
e9_printf("\n*** Descomposición de DV Asignada ***\n");
e9_printf("[br0285] Dirección Virtual Asignada: 0x%08X\n", dv_asignada);

```

```
e9_printf("[br0285] VPN1: 0x%03X\n", vpn1);
e9_printf("[br0285] VPN2: 0x%03X\n", vpn2);
e9_printf("[br0285] Offset: 0x%03X\n", offset);
}
```

5.Traducción dirección específica



```
#include "e9_system.h" /* e9_printf */
#include "kernel.h"
#include <sys/vm.h> /* I386_VM_PRESENT, ... , I386_CR0_PG */

typedef unsigned pte_t;
typedef unsigned pde_t;

#define BIOS_DATE 0x00FFFF5
#define CEIL_DIV(x, y) (((x) + (y) - 1) / (y))
#define MEM_SIZE_MB 128
#define MB_TO_BYTE (1024 * 1024)
#define NUM_PAGS_VM (MEM_SIZE_MB * MB_TO_BYTE) / PAGE_SIZE
#define NUM_PAGS_TP CEIL_DIV(NUM_PAGS_VM * sizeof(pte_t), PAGE_SIZE)
#define NUM_PAGS_DP CEIL_DIV(NUM_PAGS_TP * sizeof(pde_t), PAGE_SIZE)
#define I386_VM_PT_ENT_SHIFT 12
#define I386_VM_PD_ENT_SHIFT 22
#define I386_VM_PT_ENT_MASK 0x3FF
#define I386_VM_OFFSET_MASK 0xFFF
#define NUM_PAGS_TPB 1
```



```

pde_t rm[(NUM_PAGS_TP + NUM_PAGS_DP + NUM_PAGS_TPB + 1) * PAGE_SIZE / sizeof(pde_t)];

void setup_paging(void)
{
    int i;
    unsigned marco_tp, m, cr0, marco_dp;
    pde_t* tp;
    pde_t* dp;
    pde_t* tpb;
    unsigned dv_asignada = 0x0C83CFF5;
    unsigned vpn1, vpn2, offset;

    e9_printf("\n***Yzan Martinez Manzano***\n");

    vpn1 = (dv_asignada >> I386_VM_PD_ENT_SHIFT) & I386_VM_PT_ENT_MASK;
    vpn2 = (dv_asignada >> I386_VM_PT_ENT_SHIFT) & I386_VM_PT_ENT_MASK;
    offset = dv_asignada & I386_VM_OFFSET_MASK;

    e9_printf("[br0285] Dirección Virtual Asignada: 0x%08X\n", dv_asignada);
    e9_printf("[br0285] VPN1: 0x%03X\n", vpn1);
    e9_printf("[br0285] VPN2: 0x%03X\n", vpn2);
    e9_printf("[br0285] Offset: 0x%03X\n", offset);

    tp = (pde_t*) (((unsigned)rm) + PAGE_SIZE - 1) & ~(PAGE_SIZE - 1));
    dp = (pde_t*) (((unsigned)tp) + (NUM_PAGS_VM * sizeof(pte_t)));
    tpb = (pde_t*) (((unsigned)dp) + (NUM_PAGS_DP * sizeof(pde_t)));

    e9_printf("[br0285] Dir. tpb(abs): 0x%08X\n", ((unsigned)tpb) + kinfo.data_base);

    for (i = 0; i < NUM_PAGS_TP; i++) {
        m = marco_tp + i;
        dp[i] = (m << I386_VM_PT_ENT_SHIFT) | I386_VM_PRESENT | I386_VM_WRITE |
I386_VM_USER;
    }

    for (i = 0; i < I386_VM_PT_ENTRIES; i++) {
        tpb[i] = 0x00000000;
    }

    unsigned dpe_index = vpn1;
    dp[dpe_index] = ((unsigned)tpb - kinfo.data_base) | I386_VM_PRESENT | I386_VM_WRITE |
I386_VM_USER;
    e9_printf("[br0285] DPE[%d]: 0x%08X\n", dpe_index, dp[dpe_index]);

    unsigned pte_index = vpn2;
    tpb[pte_index] = (BIOS_DATE & ~(PAGE_SIZE - 1)) | I386_VM_PRESENT | I386_VM_WRITE |
I386_VM_USER;
    e9_printf("[br0285] PTE[%d]: 0x%08X\n", pte_index, tpb[pte_index]);
}

```